



NutriFlow

Spring Boot 後端架構設計文件

技術設計文件 v1.0

框架	Spring Boot 3.x + Java 21
資料庫	MySQL 8.0 + Redis 7
ORM	Spring Data JPA + Hibernate
安全	Spring Security + JWT
AI 整合	Anthropic Claude API (後端代理)
文件日期	2026 年 4 月

1. 系統架構總覽

1.1 整體分層架構

架構分層 (由上到下)	
Vue 3 前端 (Port 5173)	
↓ HTTPS / REST + JSON	
Spring Boot API Server (Port 8080)	

		Controller Layer (路由、請求/回應處理)	
		Service Layer (業務邏輯)	
		Repository Layer (資料存取 JPA)	
		Integration Layer (Claude AI / 外部 API)	
	MySQL 8.0	Redis 7 (快取 / Session)	

1.2 專案目錄結構

src/main/java/com/nutriflow/	
nutriflow/	
	config/ # Spring 設定 (Security、CORS、Redis...)
	controller/ # REST Controllers (@RestController)
	AuthController.java
	UserController.java
	MenuController.java
	MealController.java
	FoodController.java
	AiController.java
	WeightController.java
	service/ # 業務邏輯層 (@Service)
	AuthService.java
	UserService.java
	MenuService.java
	MealService.java
	FoodService.java
	AiProxyService.java
	NutritionService.java
	WeightService.java
	repository/ # JPA Repositories (@Repository)
	entity/ # JPA Entities (@Entity)
	dto/ # 請求/回應資料傳輸物件
	request/ # 前端送來的 Request DTO
	response/ # 回傳給前端的 Response DTO

└─ exception/	# 自訂例外與全域處理
└─ security/	# JWT 相關 (Filter、Provider)
└─ integration/	# 外部 API 整合 (Claude、OpenFoodFacts)
└─ util/	# 工具類 (TDEECalculator、NutritionUtils)

2. 依賴設定 (pom.xml)

2.1 核心依賴

依賴	Group / Artifact	用途
Spring Boot Starter Web	spring-boot-starter-web	REST API、內嵌 Tomcat
Spring Boot Starter Security	spring-boot-starter-security	JWT 驗證、授權
Spring Boot Starter Data JPA	spring-boot-starter-data-jpa	ORM、資料庫操作
Spring Boot Starter Validation	spring-boot-starter-validation	請求參數驗證 (@Valid)
Spring Boot Starter Data Redis	spring-boot-starter-data-redis	Redis 快取
MySQL Connector/J	com.mysql:mysql-connector-j	MySQL JDBC 驅動
JWT	io.jsonwebtoken:jjwt-api	JWT 生成與驗證
MapStruct	org.mapstruct:mapstruct	Entity ↔ DTO 自動轉換
Lombok	org.projectlombok:lombok	減少樣板程式碼
WebClient (WebFlux)	spring-boot-starter-webflux	呼叫外部 API (Claude、OpenFoodFacts)
SpringDoc OpenAPI	org.springdoc:springdoc-openapi-starter-webmvc-ui	Swagger UI 自動文件
Spring Boot Starter Test	spring-boot-starter-test	JUnit 5 + Mockito

2.2 application.yml 設定

```
// application.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/nutriflow?useSSL=false&serverTimezone=Asia/Taipei
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 10
      connection-timeout: 30000
```

```
jpa:
  hibernate:
    ddl-auto: validate          # 正式環境用 validate，開發用 update
    show-sql: false
    open-in-view: false        # 避免 N+1 問題

data:
  redis:
    host: ${REDIS_HOST:localhost}
    port: 6379
    password: ${REDIS_PASSWORD:}

jwt:
  secret: ${JWT_SECRET}        # 至少 256 bits
  expiration: 86400000          # 24 小時 (ms)
  refresh-expiration: 604800000 # 7 天

anthropic:
  api-key: ${ANTHROPIC_API_KEY} # 後端代理，前端不持有
  base-url: https://api.anthropic.com
  model: claude-sonnet-4-20250514
  max-tokens: 4000

open-food-facts:
  base-url: https://world.openfoodfacts.org/api/v2
  cache-ttl: 86400              # 快取 24 小時
```

3. 安全設計（Spring Security + JWT）

3.1 JWT 認證流程

認證流程

1. 前端 POST /api/auth/login → { email, password }
2. AuthService 驗證帳號密碼（BCrypt）
3. 產生 accessToken（24h）+ refreshToken（7d）
4. 前端儲存 accessToken（記憶體）+ refreshToken（HttpOnly Cookie）
5. 後續請求帶 Authorization: Bearer {accessToken}
6. JwtAuthFilter 攔截 → 驗證 Token → 設定 SecurityContext
7. AccessToken 過期 → 自動呼叫 POST /api/auth/refresh

3.2 SecurityConfig

```
// SecurityConfig.java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(sm ->
                sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .requestMatchers("/v3/api-docs/**", "/swagger-ui/**").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(customAuthEntryPoint)
            )
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }
}
```

4. REST API 設計

4.1 API 端點總覽

Method	路徑	說明	Auth
POST	/api/auth/register	註冊帳號	✗
POST	/api/auth/login	登入，回傳 tokens	✗
POST	/api/auth/refresh	刷新 Access Token	✗
POST	/api/auth/logout	登出（清除 refresh cookie）	☑
GET	/api/users/me	取得當前使用者資料	☑

Method	路徑	說明	Auth
PUT	/api/users/me	更新基本資料 / 目標設定	☑
GET	/api/users/me/cat	取得貓咪設定	☑
PUT	/api/users/me/cat	更新貓咪品種與校正資料	☑
GET	/api/menus/week	取得本週菜單	☑
POST	/api/menus/generate	AI 生成週菜單	☑
PATCH	/api/menus/week/{day}/{meal}	更新單格餐點（多食物）	☑
POST	/api/meals	記錄餐點	☑
GET	/api/meals	查詢餐點（?date=&page=）	☑
PUT	/api/meals/{id}	更新餐點記錄	☑
DELETE	/api/meals/{id}	刪除餐點記錄	☑
GET	/api/foods/search	食物搜尋（?q=&store=）	☑
GET	/api/foods/official/{store}	官方連鎖店營養資料	☑
POST	/api/ai/analyze-photo	拍照估算熱量（Claude Vision）	☑
POST	/api/ai/suggest-meals	AI 推薦替換餐點	☑
POST	/api/ai/menu-comment	小橘的飲食評語	☑
POST	/api/weight	記錄體重	☑
GET	/api/weight	取得體重歷史	☑
GET	/api/nutrition/daily	取得今日營養統計	☑
GET	/api/nutrition/weekly	取得本週營養趨勢	☑

4.2 統一回應格式

```
// ApiResponse 格式
// 成功回應
{
  "success": true,
  "data": { ... },
  "message": "操作成功",
  "timestamp": "2026-04-26T10:30:00+08:00"
}

// 錯誤回應
{
  "success": false,
  "error": {
```

```
    "code": "FOOD_NOT_FOUND",
    "message": "找不到該食物資料",
    "details": null
  },
  "timestamp": "2026-04-26T10:30:00+08:00"
}

// 分頁回應
{
  "success": true,
  "data": {
    "content": [...],
    "page": 0,
    "size": 20,
    "totalElements": 150,
    "totalPages": 8
  }
}
```

5. JPA Entity 設計

5.1 Entity 關聯圖

Entity 關聯（簡化）	
User (1)	———— (1) CatProfile
├ (1:N) WeekMenu	—— (1:N) MealCell ——— (1:N) MealItem
├ (1:N) MealLog	—— (1:N) MealLogItem
├ (1:N) WeightRecord	
└ (1:N) FoodPreference	
FoodDatabase (獨立主資料表)	
├ OfficialNutrition (連鎖店官方資料)	
└ UserCustomFood (使用者自訂食物)	

5.2 User Entity

```
// User.java
@Entity
@Table(name = "users")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class User {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String passwordHash;

    // 基本資料
    private String gender;          // "male" | "female"
    private Integer age;
    private Double weight;          // kg
    private Double height;          // cm
    private String activityLevel;

    // 目標設定
    private Boolean hasWeightGoal;
    private Double targetWeight;
    private Integer goalWeeks;

    // 系統欄位
    private Boolean isSetupComplete = false;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    // Relations
    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL)
    private CatProfile catProfile;
}
```

5.3 MealCell + MealItem Entity（多食物支援）

```
// MealCell.java + MealItem.java
@Entity @Table(name = "meal_cells")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class MealCell {
```



```

@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "week_menu_id")
private WeekMenu weekMenu;

private Integer dayIndex;    // 0=週一 ... 6=週日
private Integer mealIndex;   // 0=早餐 1=午餐 2=晚餐

@Enumerated(EnumType.STRING)
private MealType mealType;   // EATING_OUT | HOME_COOK

private Boolean logged = false;
private String note;
private LocalTime logTime;

@Enumerated(EnumType.STRING)
private DataSource source;    // AI | MANUAL | PHOTO

// 一格餐點包含多個食物項目
@OneToMany(mappedBy = "mealCell", cascade = CascadeType.ALL, orphanRemoval = true)
@OrderBy("sortOrder ASC")
private List<MealItem> items = new ArrayList<>();
}

@Entity @Table(name = "meal_items")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class MealItem {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "meal_cell_id")
    private MealCell mealCell;

    private String foodName;
    private Integer kcal;
    private Double protein;    // g
    private Double carbs;     // g
    private Double fat;        // g
    private Double sodium;     // mg
    private Integer sortOrder;
}

```

6. Service 層設計

6.1 MenuService — AI 菜單生成

```
// MenuService.java
@Service
@RequiredArgsConstructor
public class MenuService {

    private final AiProxyService aiProxyService;
    private final WeekMenuRepository weekMenuRepo;
    private final MealCellRepository mealCellRepo;
    private final UserService userService;

    public WeekMenuResponse generateMenu(Long userId, MenuGenerateRequest req) {
        User user = userService.findById(userId);

        // 計算 TDEE 與目標熱量
        int tdee = TDEECalculator.calculate(user);
        int targetKcal = req.getKcalTarget() != null ? req.getKcalTarget() : tdee;
        int targetProt = req.getProteinTarget() != null
            ? req.getProteinTarget()
            : (int)(user.getWeight() * 1.2);

        // 呼叫 AI 生成
        String prompt = buildMenuPrompt(user, targetKcal, targetProt, req);
        String aiJson = aiProxyService.callClaude(prompt);

        // 解析並儲存
        WeekMenuDto dto = parseMenuJson(aiJson);
        WeekMenu menu = saveWeekMenu(userId, dto);

        return WeekMenuMapper.toResponse(menu);
    }

    public void updateMealCell(Long userId, int dayIdx, int mealIdx,
                               UpdateMealCellRequest req) {
        MealCell cell = mealCellRepo
            .findByWeekMenuUserIdAndDayIndexAndMealIndex(userId, dayIdx, mealIdx)
            .orElseThrow(() -> new ResourceNotFoundException("MealCell"));

        // 清除舊的 MealItem，重建新的（支援多食物）
        cell.getItems().clear();
        req.getFoods().forEach((food, idx) -> {
            cell.getItems().add(MealItem.builder()
                .foodName(food.getName())
                .kcal(food.getKcal())
                .sortOrder(idx)
                .mealCell(cell)
                .build());
        });
    }
}
```

```

        cell.setLogged(!req.getFoods().isEmpty());
        cell.setMealType(req.getMealType());
        cell.setNote(req.getNote());
        mealCellRepo.save(cell);
    }
}

```

6.2 AiProxyService — Claude API 代理

```

// AiProxyService.java
@Service
@RequiredArgsConstructor
public class AiProxyService {

    private final WebClient webClient;
    private final AiProperties aiProps;

    // 文字 prompt → Claude → 回傳字串
    public String callClaude(String prompt) {
        ClaudeRequest request = ClaudeRequest.builder()
            .model(aiProps.getModel())
            .maxTokens(aiProps.getMaxTokens())
            .messages(List.of(new ClaudeMessage("user", prompt)))
            .build();

        return webClient.post()
            .uri(aiProps.getBaseUrl() + "/v1/messages")
            .header("x-api-key", aiProps.getApiKey())
            .header("anthropic-version", "2023-06-01")
            .bodyValue(request)
            .retrieve()
            .bodyToMono(ClaudeResponse.class)
            .map(r -> r.getContent().get(0).getText())
            .block();
    }

    // 圖片 (base64) + prompt → Claude Vision → 分析結果
    public PhotoAnalysisResult analyzePhoto(String base64Image,
        String mediaType,
        double knuckleWidthMm) {

        String prompt = buildPhotoPrompt(knuckleWidthMm);
        // 組裝多模態 message (image block + text block)
        ClaudeRequest req = buildVisionRequest(base64Image, mediaType, prompt);
        String json = callClaudeRaw(req);
        return parsePhotoResult(json);
    }
}

```

6.3 FoodService — 三層查詢邏輯

```
// FoodService.java
@Service
@RequiredArgsConstructor
public class FoodService {

    private final OfficialNutritionRepository officialRepo;
    private final OpenFoodFactsClient offClient;
    private final AiProxyService aiService;
    private final RedisTemplate<String, Object> redis;

    public FoodSearchResponse search(String query, String storeName) {
        // 1. 檢查 Redis 快取
        String cacheKey = "food:" + query + ":" + storeName;
        Object cached = redis.opsForValue().get(cacheKey);
        if (cached != null) return (FoodSearchResponse) cached;

        FoodSearchResponse result;
        DataSourceType source;

        // 2. 官方資料庫（連鎖店）
        if (storeName != null && isKnownChain(storeName)) {
            var official = officialRepo.findByStoreAndItem(storeName, query);
            if (official.isPresent()) {
                result = mapToResponse(official.get(), DataSourceType.OFFICIAL);
                cacheResult(cacheKey, result, Duration.ofDays(7));
                return result;
            }
        }

        // 3. Open Food Facts API
        var offResult = offClient.search(query);
        if (offResult.isPresent()) {
            result = mapToResponse(offResult.get(), DataSourceType.OPEN_FOOD_FACTS);
            cacheResult(cacheKey, result, Duration.ofDays(1));
            return result;
        }

        // 4. Claude AI 估算 (fallback)
        result = aiService.estimateNutrition(query, storeName);
        result.setSource(DataSourceType.AI_ESTIMATE);
        return result;
    }
}
```

7. 例外處理

7.1 全域例外處理器

```
// GlobalExceptionHandler.java
@RestControllerAdvice
public class GlobalExceptionHandler {

    // 資源找不到
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiResponse<?>> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(404)
            .body(ApiResponse.error("NOT_FOUND", ex.getMessage()));
    }

    // 請求驗證失敗
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResponse<?>> handleValidation(...) {
        List<String> errors = ex.getBindingResult().getFieldErrors()
            .stream().map(e -> e.getField() + ": " + e.getDefaultMessage())
            .toList();
        return ResponseEntity.status(400)
            .body(ApiResponse.error("VALIDATION_FAILED", errors.toString()));
    }

    // AI 服務不可用
    @ExceptionHandler(AiServiceException.class)
    public ResponseEntity<ApiResponse<?>> handleAiError(AiServiceException ex) {
        return ResponseEntity.status(503)
            .body(ApiResponse.error("AI_SERVICE_UNAVAILABLE", ex.getMessage()));
    }

    // 其他未預期例外
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse<?>> handleGeneral(Exception ex) {
        log.error("Unexpected error", ex);
        return ResponseEntity.status(500)
            .body(ApiResponse.error("INTERNAL_ERROR", "伺服器發生錯誤"));
    }
}
```

7.2 自訂例外類別

例外類別	HTTP Status	使用情境
ResourceNotFoundException	404	找不到 User、MealCell 等資源
DuplicateEmailException	409	註冊時 email 已存在
InvalidCredentialsException	401	帳號密碼錯誤
TokenExpiredException	401	JWT Token 過期

例外類別	HTTP Status	使用情境
AiServiceException	503	Claude API 呼叫失敗
PhotoAnalysisException	422	圖片分析失敗（格式不支援等）
CalorieTargetException	400	設定的熱量目標低於安全下限

8. Redis 快取策略

快取 Key 格式	TTL	說明
food:{query}:{store}	24 小時	食物搜尋結果（官方 / Open Food Facts）
official:{store}:{item}	7 天	連鎖店官方營養資料
menu:week:{userId}:{weekStart}	1 小時	週菜單快取（編輯後失效）
nutrition:daily:{userId}:{date}	10 分鐘	每日營養統計
user:profile:{userId}	5 分鐘	使用者資料快取
refresh:token:{userId}	7 天	Refresh Token 黑名單管理

8.1 快取注解使用

```
// Cache 注解範例
// 讀取時自動快取
@Cacheable(value = "userProfile", key = "#userId")
public UserProfileResponse getProfile(Long userId) { ... }

// 更新後清除快取
@CacheEvict(value = "userProfile", key = "#userId")
public void updateProfile(Long userId, UpdateProfileRequest req) { ... }

// 更新後同步更新快取
@CachePut(value = "userProfile", key = "#userId")
public UserProfileResponse updateAndCache(Long userId, ...) { ... }
```

9. 營養計算工具（Utils）

9.1 TDEE Calculator

```
// TDEECalculator.java
public class TDEECalculator {

    private static final Map<String, Double> ACTIVITY_FACTORS = Map.of(
        "sedentary",    1.20,
        "light",        1.375,
        "moderate",     1.55,
        "active",       1.725,
        "very_active",  1.90
    );

    // Mifflin-St Jeor 公式
    public static int calculate(User user) {
        double bmr;
        if ("female".equals(user.getGender())) {
            bmr = 10 * user.getWeight() + 6.25 * user.getHeight()
                - 5 * user.getAge() - 161;
        } else {
            bmr = 10 * user.getWeight() + 6.25 * user.getHeight()
                - 5 * user.getAge() + 5;
        }
        double factor = ACTIVITY_FACTORS.getOrDefault(
            user.getActivityLevel(), 1.55);
        return (int) Math.round(bmr * factor);
    }

    public static int calcDailyDeficit(double currentKg,
                                       double targetKg, int weeks) {
        if (currentKg <= targetKg || weeks <= 0) return 0;
        int deficit = (int) Math.round((currentKg - targetKg) * 7700 / (weeks * 7));
        return Math.min(deficit, 500); // 上限 500 kcal/天
    }

    public static int safeTargetCals(int tdee, int deficit,
                                       String gender) {
        int min = "female".equals(gender) ? 1200 : 1500;
        return Math.max(min, tdee - deficit);
    }
}
```

9.2 NutritionAdviceEngine

```
// NutritionAdviceEngine.java
public class NutritionAdviceEngine {

    public List<NutritionTip> analyze(DailyNutrition actual,
                                       UserProfile profile) {
        List<NutritionTip> tips = new ArrayList<>();
        int meals = profile.getMealCount(); // 2 or 3
    }
}
```

```

// 脂肪佔比 > 35%
double fatPct = actual.getFatKcal() / actual.getTotalKcal();
if (fatPct > 0.35)
    tips.add(new NutritionTip(TipType.HIGH_FAT,
        "油脂攝取偏高，建議選清蒸、水煮或減少油炸"));

// 蛋白質 < 體重 × 0.8g
double minProtein = profile.getWeight() * 0.8;
if (actual.getProtein() < minProtein)
    tips.add(new NutritionTip(TipType.LOW_PROTEIN,
        "蛋白質不足，建議增加雞肉、蛋、豆腐等"));

// 鈉 > 2000mg/天
if (actual.getSodium() > 2000)
    tips.add(new NutritionTip(TipType.HIGH_SODIUM,
        "鈉含量偏高，注意醬料與加工食品的攝取"));

if (tips.isEmpty())
    tips.add(new NutritionTip(TipType.BALANCED, "營養均衡，繼續保持！"));

return tips;
}
}

```

10. 開發環境建置

10.1 快速啟動

1. 確認 Java 21 + Maven 3.9 已安裝
2. 建立 MySQL 資料庫：CREATE DATABASE nutriflow CHARACTER SET utf8mb4;
3. 複製 .env.example → .env，填入 DB / JWT / Anthropic API Key
4. mvn spring-boot:run 或 執行 NutriflowApplication.java
5. Swagger UI：http://localhost:8080/swagger-ui.html

10.2 環境變數清單

變數名稱	說明	範例值
DB_HOST	MySQL 主機	localhost
DB_PORT	MySQL 埠	3306
DB_NAME	資料庫名稱	nutriflow

變數名稱	說明	範例值
DB_USERNAME	資料庫帳號	nutriflow_user
DB_PASSWORD	資料庫密碼	(自行設定)
REDIS_HOST	Redis 主機	localhost
REDIS_PASSWORD	Redis 密碼	(開發可留空)
JWT_SECRET	JWT 簽名密鑰 (256 bits+)	(隨機產生)
ANTHROPIC_API_KEY	Claude API Key	sk-ant-xxxxx

10.3 Flyway 資料庫版本管理

- 使用 Flyway 管理 Schema 變更，不使用 ddl-auto: update
- Migration 腳本放於 src/main/resources/db/migration/
- 命名規則：V{版本}__{描述}.sql，例：V1__init_schema.sql
- 開發環境可執行 mvn flyway:repair 修復 checksum 不一致

後端開發注意事項

1. Claude API 呼叫統一透過 AiProxyService，禁止在 Controller 層直接呼叫
2. 所有外部 API 呼叫需加 @Retryable (最多重試 3 次，指數退避)
3. 圖片 base64 不寫入資料庫，分析完成後立即丟棄
4. 敏感欄位 (passwordHash、token) 不得出現在 Response DTO
5. 所有 Controller 方法需加 @PreAuthorize 或透過 SecurityConfig 管控
6. 使用 @Transactional 時注意 ReadOnly = true 可提升查詢效能
7. 單元測試覆蓋率目標：Service 層 > 80%，Utils 100%